

Computer Architectures

Exam of 18/02/2026

B1

Please read the following notes before proceeding

- 1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project inside the exam folder on the Z drive (the same folder where you found this document). **Projects saved elsewhere will not be collected and will be permanently lost.**
- 2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
- 3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
; to do: add code
MOV r0, #10      ; example of a return value
BX LR
ENDP
```

- 4) To earn points on all exercises, it is recommended that you write at least some code for each.
- 5) **The final project must run on the actual board (e.g., button debouncing is required).**

Special characters with Italian keyboards

- Square brackets []
Left bracket: ATL GR + [
Right bracket: ALT GR +]
- Curly brackets { }
Left bracket: LSHIFT + ALT GR + [
Right bracket: LSHIFT + ALT GR +]
- Hash # : ALT GR + à

Bitwise operations in C

Left shift by N bits = $A \ll N$
Right shift by N bits = $A \gg N$

A XOR B = $A \wedge B$
A AND B = $A \& B$ (&& is the logical AND)
A OR B = $A | B$ (|| is the logical OR)

Computer Architectures

Exam of 18/02/2026

B1

Question 1 (8 points)

The Hofstadter Q sequence is defined as follows:

$$Q(1) = Q(2) = 1$$

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)), \text{ for } n > 2$$

The first terms of the sequence are:

1, 1, 2, 3, 3, 4, 5, 5, 6, 6, 6, 8, 8, 8, 10, 9, 10, 11, 11, 12, 12, 12, 12, 16, 14, 14, 16, 16, 16, 20, 17, 17, 20, 21, 19, 20, 22, 21, 22, 23, 23, 24, 24, 24, 24, 24, 32, 24, 25, 30, 28, 26, 30, 30, 28, 32, 30, 32, 32, 32, 32, 40, 33, 31, 38, 35, 33, 39, 40, 37, 38, 40, 39

Write the `HofstadterQ` subroutine in ARM assembly language. You must follow this prototype:

unsigned int HofstadterQ(unsigned int* v, int dim);

The subroutine receives:

- the address of an uninitialized read-write data area
- the number of elements *dim*.

The subroutine must fill the data area with the first *dim* elements of the Hofstadter Q sequence.

Each element must be stored as a word (you must reserve sufficient space when allocating the read-write data area).

The subroutine must return the maximum value among the first *dim* elements of the Hofstadter Q sequence.

Suggestion. Avoid a recursive implementation of the sequence. Instead, store each term in the data area, starting with $Q(1)$ and $Q(2)$. When computing $Q(n)$, access the data area to retrieve the previously computed elements $Q(n-1)$, $Q(n-2)$, etc.

Question 2 (10 points)

In this part, you need 3 timers, referred to in the text as timer *A*, timer *B*, and timer *C*. You are free to choose which timer to use as timer *A*. For timer *B* and *C*, it is suggested to use two standard Capture/Compare timers.

Some actions depend on whether a timer is running. You can use a flag or check the appropriate timer register to determine whether a timer is active.

Add the following functionalities to your project. The goal of this exercise is to generate a musical representation of the Hofstadter Q sequence by mapping its values to sound frequency and duration.

1) In the `main` function, call the `HofstadterQ` subroutine passing an uninitialized integer array *v* of size 1000. As specified in the first question, the subroutine fills the array and returns Q_{max} . Initialize the D/A Converter so that its output is connected to the input of the external speaker (pin P0.26).

Finally, start timer *A* with a period of 50 ms.

2) When timer *A* generates an interrupt, check whether timer *B* and timer *C* are running. If either of them is running, the interrupt handler of timer *A* must terminate immediately.

Otherwise increment an index *i*. This index is used to access the *i*-th element of the Hofstadter Q sequence. For example, at the first interrupt, you access the value $Q(1)$, since timer *B* and timer *C*

Computer Architectures

Exam of 18/02/2026

B1

are initially stopped. Note that $Q(1)$ is stored in $v[0]$, $Q(2)$ in $v[1]$, and in general $Q(n)$ is stored in $v[n-1]$. When i reaches 1000, you access the value $Q(1000)$ and then stop the timer A .

Each time you increment the index i , compute two thresholds according to the following formula:

$$threshold = \frac{1}{k} \left(P_{max} - \frac{Q(i) * (P_{max} - P_{min})}{Q_{max}} \right)$$

The formula computes a floating-point value, but each threshold must be cast to an unsigned integer.

For the first threshold, use $P_{max} = 5351$, $P_{min} = 1062$, $k = 1$.

For the second threshold, use $P_{max} = 40,000,000$, $P_{min} = 625,000$, $k = 5$. Here, to avoid overflow due to intermediate multiplications, carefully choose the order of evaluation: it is recommended to compute $((P_{max} - P_{min}) / Q_{max}) \cdot Q(i)$.

Then, start timer B with the first threshold, so that when its timer counter reaches the threshold, an interrupt is generated and the timer counter is **reset**.

Start timer C with the second threshold, so that when its timer counter reaches the threshold, an interrupt is generated and the timer counter is **stopped and reset**.

3) Define the following array, which contains samples of a sinusoidal waveform:

```
uint16_t SinTable[45] = {410, 467, 523, 576, 627, 673, 714, 749, 778, 799, 813, 819, 817, 807, 789, 764, 732, 694, 650, 602, 550, 495, 438, 381, 324, 270, 217, 169, 125, 87, 55, 30, 12, 2, 0, 6, 20, 41, 70, 105, 146, 193, 243, 297, 353};
```

When timer B generates an interrupt, increment an index j . This index is used to access the j -th element of *SinTable*. Use that element to feed the DAC.

If the index j reaches 45, you must reset it (without stopping the timer B).

4) When timer C generates an interrupt, stop timer B . In other words, timer C determines the duration of the note played by timer B .